



Università degli Studi di Napoli Federico II
Corso di Laurea Triennale in Informatica

Documentazione Hackathon Basi Di Dati

Mario Palladino N86005143

Anno Accademico 2024/2025

Introduzione

Questa documentazione ha l'obiettivo di:

1. Esporre l'analisi dei requisiti condotta a partire dal problema proposto dal cliente
2. Illustrare la progettazione concettuale sviluppata sulla base dell'analisi effettuata
3. Presentare lo schema logico del modello effettuata al fine di permetterne l'implementazione
4. Descrivere la base di dati concreta, traducendo gli elementi determinati nelle fasi precedenti in tabelle fisiche

1 Specifica e Analisi dei Requisiti

1.1 Definizione del dominio

I requisiti sono specificati all'interno della traccia dell'esercizio

Un hackathon, ovvero una "maratona di hacking", è un evento durante il quale team di partecipanti si sfidano per progettare e implementare nuove soluzioni basate su una certa tecnologia o mirate a un certo ambito applicativo.

Ogni hackathon ha un titolo identificativo, si svolge in una certa sede e in un certo intervallo di tempo (solitamente 2 giorni) e ha un organizzatore specifico (registrato alla piattaforma). L'organizzatore seleziona un gruppo di giudici (selezionati tra gli utenti della piattaforma, invitandoli). Infine, l'organizzatore apre le registrazioni, che si chiuderanno 2 giorni prima dell'evento. Ogni evento avrà un numero massimo di iscritti e una dimensione massima del team.

Durante il periodo di registrazione, gli utenti possono registrarsi per l'Hackathon di loro scelta (eventualmente registrandosi sulla piattaforma se non lo hanno già fatto). Una volta iscritti, gli utenti possono formare team. I team diventano definitivi quando si chiudono le iscrizioni. All'inizio dell'hackathon, i giudici pubblicano una descrizione del problema da affrontare.

Durante l'hackathon, i team lavorano separatamente per risolvere il problema e devono caricare periodicamente gli aggiornamenti sui "progressi" sulla piattaforma come documento, che può essere esaminato e commentato dai giudici. Alla fine dell'hackathon, ogni giudice assegna un voto (da 0 a 10) a ciascun team e la piattaforma, dopo aver acquisito tutti i voti, pubblica le classifiche dei team.

1.2 Obiettivi del Sistema

A partire dalla traccia fornita il sistema da progettare deve consentire:

- La gestione completa del ciclo di vita di un hackathon
- La registrazione e autenticazione degli utenti sulla piattaforma
- La formazione e gestione dei team partecipanti
- Il caricamento e la valutazione dei documenti prodotti
- La generazione automatica delle classifiche finali

1.3 Analisi dei Requisiti

1.3.1 Requisiti Informativi

Il sistema prevede la realizzazione di una piattaforma atta all'organizzazione e alla gestione di una maratona di hacking in cui team di partecipanti collaborano per sviluppare soluzioni innovative in un tempo limitato, solitamente 48 ore. Ogni hackathon è unico e prevede una serie di caratteristiche ben definite che ne regolano l'organizzazione:

- **Organizzazione dell'Hackathon:**
L'**Hackathon** viene creato dalla figura dell'**Organizzatore**, un **Utente** registrato sulla piattaforma, che ne definisce i dettagli fondamentali: un *titolo* identificativo, la *sede* in cui si svolgerà, la *durata* (tipicamente due giorni), e parametri operativi come il *numero massimo di partecipanti* e la *dimensione massima dei team*.
- **Selezione dei Giudici:**
Una volta configurato l'evento, l'**Organizzatore** seleziona un gruppo di **Giudici** tra gli **Utenti** già presenti nella piattaforma, invitandoli a valutare i progetti presentati dai **Partecipanti**.
- **Registrazione e formazione dei Team:**
Le iscrizioni **all'Hackathon** vengono aperte **dall'Organizzatore** e rimangono disponibili fino a 48 ore prima dell'inizio dell'evento, momento in cui i Team formati diventano definitivi. In questo periodo, gli **Utenti** possono registrarsi singolarmente o formare **Team**. Se un **Utente** non è ancora registrato sulla piattaforma, può creare un account in fase di iscrizione.

- Svolgimento **dell'Hackathon**

All'inizio della competizione, i **Giudici** pubblicano la *descrizione del problema da risolvere*, che servirà come base per lo sviluppo dei progetti. Durante l'evento, i **Team** lavorano in modo indipendente ma sono tenuti a caricare periodicamente aggiornamenti sui loro progressi sotto forma di **documenti** successivamente valutati dai **Giudici**.

- Valutazione e classifica finale

Al termine **dell'Hackathon**, ogni **Giudice** assegna un *voto* a ciascun **Team** in base alla qualità della soluzione proposta. La piattaforma raccoglie automaticamente tutti i voti e pubblica una classifica finale resa disponibile a tutti gli **Utenti**.

1.3.2 Vincoli Individuati

Per garantire il corretto funzionamento della sistema, sono state imposte le seguenti regole:

- Un **Utente** può assumere più di un ruolo, ma **non** in una *singola* competizione
- Un **Utente** deve essere registrato **all'Hackathon** prima di poter prendere parte ad un **Team**
- Un **Utente** può unirsi a più **Team**, ma **non** in in *singolo Hackathon*
- Un **Team** pu`o essere formato anche da un solo **Partecipante**
- Un gruppo di **Giudici**, per essere considerato valido, deve essere composto da almeno 2 membri
- Un **Hackathon**, per essere considerato valido, deve prevedere almeno 2 **Team**
- Il *Voto* finale deve essere compreso tra 0 e 10
- Prima di poter dare un *Voto* ad un **Team**, un **Giudice** deve *esaminare* ogni documento prodotto da quel team.
- Un **Partecipante** deve rispettare i vincoli temporali impostegli **dall'Organizzatore** per poter effettuare le operazioni che desidera.
- Per essere valutati correttamente, i **Team** devono produrre almeno 1 **Documento**

1.3.3 Requisiti Funzionali

Il sistema per la gestione degli hackathon deve soddisfare una serie di funzionalità a volte a supportare il ciclo di vita completo degli eventi di programmazione competitiva, dalla creazione dell'evento fino alla pubblicazione delle classifiche finali.

Gestione degli utenti

Registrazione di nuovi utenti sulla piattaforma, specificando:

- Credenziali di accesso
- Dati anagrafici

Autenticazione e gestione del profilo utente.

Gestione degli hackathon

Creazione di un nuovo hackathon da parte di un organizzatore, specificando:

- Titolo identificativo
- Sede dell'evento
- Intervallo temporale (tipicamente 2 giorni)
- Numero massimo di iscritti
- Dimensione massima del team

Visualizzazione e ricerca di hackathon per:

- Titolo
- Sede
- Data
- Organizzatore

Gestione dei giudici

Selezione e invito di giudici tra gli utenti registrati

Pubblicazione della descrizione del problema da affrontare all'inizio dell'hackathon.

Consultazione e commento dei progressi caricati dai team.

Gestione delle registrazioni

Apertura delle registrazioni per un hackathon specifico.

Registrazione dei partecipanti durante il periodo di iscrizione.

Controllo del rispetto del numero massimo di iscritti.

Gestione dei team

Formazione dei team da parte dei partecipanti registrati:

- Creazione di nuovi team
- Adesione ad altri team
- Controllo del rispetto della dimensione massima del team

Finalizzazione automatica dei team alla chiusura delle iscrizioni.

Visualizzazione della composizione dei team.

Gestione dei progressi

Caricamento periodico degli aggiornamenti sui progressi:

- Upload di documenti contenenti lo stato di avanzamento

Consultazione dei progressi da parte dei giudici.

Aggiunta di commenti e feedback da parte dei giudici.

Gestione delle valutazioni

Assegnazione dei voti finali:

- Voto da 0 a 10 per ciascun team da parte di ogni giudice
- Controllo della completezza delle valutazioni
- Calcolo automatico delle medie e classifiche

Pubblicazione delle classifiche finali una volta acquisiti tutti i voti.

2 Progettazione Concettuale

2.1 Entità e Relazioni

Gli oggetti del dominio sono stati modellati attraverso l'individuazione di Entità e Relazioni, con i rispettivi attributi.

2.1.1 Entità

Utente Rappresenta tutti gli attori del sistema: - Nome - Cognome - Username - Password Partecipante Giudice Organizzatore Team Gruppi di lavoro dei partecipanti: - Nome - NumeroMembri Documento I documenti prodotti dai team: - Titolo - Contenuto - Commento	Hackathon Evento principale del sistema: - Titolo - Sede - Durata - DataInizio - DataFine - PeriodoIscrizioni - DataAperturaIscrizioni - DataChiusuraIscrizioni - MaxIscritti - MaxDimTeam - DescrizioneProblema
---	--

2.1.2 Relazioni

Le relazioni individuate che intercorrono tra le entità del sistema:

Registrazione - Data	Pubblicazione
Partecipazione	Votazione - Voto
Competizione	Selezione
Organizzazione	Esaminazione

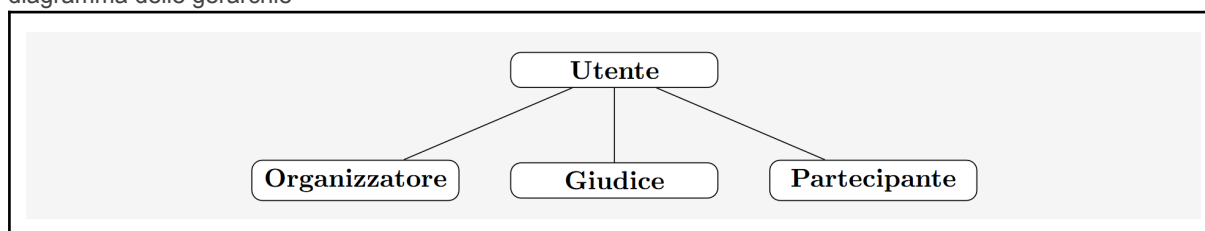
per convenzione abbiamo usato:

- Nome** -> Entità Relazione
- Nome -> Attributo dell'Entità o della Relazione
- Attributo** -> Attributo Identificativo Dell'Entità
- Attributo** -> Attributo Identificativo **Parziale** dell'Entità

2.2 Gerarchie tra Entità

Durante la progettazione concettuale è stata individuata l'entità generale **Utente**, dalla quale derivano tre entità in specializzazione: **Organizzatore**, **Giudice** e **Partecipante**. Tale gerarchia consente di modellare in maniera più accurata i diversi ruoli che gli utenti possono assumere all'interno di un **Hackathon**.

diagramma delle gerarchie



2.2.1 Vincoli della Gerarchia

I vincoli che caratterizzano la gerarchia sono:

- **Vincolo di Overlapping:** un'istanza può appartenere a *più* di una sottoclasse.
- **Vincolo di Completezza Parziale:** un'istanza della superclasse non è obbligata a far parte di una sottoclasse.

Questo vincolo riguarda la base di dati nella sua interezza ma, come specificato precedentemente, nel contesto di un singolo **Hackathon** assume la forma di un **vincolo di disgiunzione**.

Tali vincoli garantiscono il corretto funzionamento del sistema, in accordo con l'analisi dei requisiti:

- Un **Utente** "diventa" **Partecipante** se e solo se ($\Leftrightarrow$) completa la registrazione ad un **Hackathon**. Gli utenti generici non assumono automaticamente questo status.
- I **Giudici** hanno un ruolo esclusivo: possono valutare i progetti ma non iscriversi come partecipanti.
- Gli **Organizzatori** hanno accesso solo alle funzioni di gestione e non possono partecipare direttamente alla competizione.

2.3 Scelte di Progettazione Concettuale

Per rendere il sistema quanto più aderente al contesto reale, le seguenti entità e relative relazioni sono state progettate in modo da consentire la gestione dello **storico degli eventi e delle interazioni**:

- **Partecipante:** tramite le relazioni **Registrazione** e **Partecipazione** memorizziamo lo storico di partecipazioni agli hackathon passati ed attuali, senza dover vincolare gli utenti a giocare in una sola competizione alla volta.
- **Organizzatore:** con la relazione di **Organizzazione** conserviamo i record di gestione delle varie gare, evitando di avere un **Utente** vincolato ad un **Hackathon** singolo.
- **Giudice:** grazie alle relazioni **Votazione** ed **Esaminazione** conserviamo trasversalmente nei vari **Hackathon** le valutazioni emesse dai giudici, grazie al fatto che un **Team** è univocamente legato al proprio Hackathon di riferimento.
- **Selezione:** questa relazione ternaria consente di conservare le varie istanze di elezione di **Giudici** dagli **Organizzatori** anche tramite stessi **Utenti**, grazie al collegamento con **Hackathon** che consente di discernere univocamente un record dall'altro, conservando così lo storico di inviti.

Questo approccio consente di:

- mantenere una **tracciabilità temporale** delle attività
- permettere **analisi retrospettive** e la generazione di report
- garantire **scalabilità** e supporto a future evoluzioni della piattaforma

Dal punto di vista pratico sarà quindi possibile ottenere:

- per ogni **Partecipante**: a quali **Team** ha preso parte e in quali **Hackathon**
- per ogni **Organizzatore**: quali **Hackathon** ha gestito e quali **Giudici** ha invitato
- per ogni **Giudice**: l'elenco di valutazioni che ha emesso in ogni **Hackathon**

In questo modo la progettazione garantisce **coerenza, completezza e scalabilità**. E' doveroso aggiungere che su richiesta di un utente o a seguito di *manutenzione periodica* della base di dati, questi record aggiuntivi preservati per lo storico possano essere **eliminati** per garantire il funzionamento del sistema nel tempo.

Inoltre, durante questa fase di progettazione concettuale si è scelto di includere degli attributi non esplicitamente richiesti dalla traccia, al fine di meglio esprimere i concetti e le necessità che sono intrinseche in una richiesta come quella del problema in esame, in particolare:

- Nome — Cognome — Username — Password
Questi attributi sono stati inseriti nella fase concettuale per esprimere il concetto di *"utenti iscritti alla piattaforma"*, per memorizzarne le credenziali di accesso e simulare un sistema di account.
- Nome — NumeroMembri
Questi attributi sono presenti per identificare in modo più esplicativo un **Team**, presentando informazioni semplici e con poco carico sul sistema; un buon compromesso tra richiesta del cliente e chiarezza della base di dati.
- Data
Un attributo utile a dare rilevanza ai record riferiti ad una **Registrazione** da parte di un **Utente**. Memorizza la data precisa nella finestra di tempo utile per registrarsi nel quale l'**Utente** ha effettuato tale operazione.
- DataInizio — DataFine — DataAperturalscrizioni — DataChiusuralscrizioni
I suddetti attributi esprimono più chiaramente le specifiche date necessarie a tracciare la *durata* della competizione e il suo relativo *periodo utile per le registrazioni*. Inoltre, l'attributo *DataChiusuralscrizioni* potrebbe essere considerato come *derivabile*, poiché la richiesta del cliente sottolinea che le iscrizioni si chiudano **"2 giorni prima dell'evento"**. Potremmo quindi effettuare una semplice sottrazione

ed ottenere la *DataInizio*, ma si è scelto di rilassare questo vincolo per permettere al sistema una maggiore flessibilità, a discrezione **dell'Organizzatore**.

Relazione Ternaria (M:N:P)

La relazione di **Selezione** è espressa tramite una rara ma utile opzione a disposizione dei progettisti di basi di dati: una **relazione n-aria**. Questa associazione è frutto di un'estensione di quella binaria tra **Organizzatore** e **Giudice**, che non presenterebbe modo di rintracciare **l'Hackathon** di riferimento per via dell'implementazione dello *storico degli eventi*. Così facendo introduciamo un legame univoco per ogni diverso **Hackathon**, permettendo che lo stesso **Organizzatore** inviti gli stessi **Giudici** in diverse competizioni.

E' stato scelto, inoltre, di **NON reificare** l'associazione in quanto un'entità *Selezione* sarebbe stata troppo lontana da quello che si evince dal mini-world, descrivendo un aspetto poco riconducibile ad una vera e propria entità del sistema

2.4 Dizionario delle Entità

Entità	Attributi	Descrizione
Utente	- Nome - Cognome - Username - Password	Persona iscritta alla piattaforma che può prendere parte agli eventi disponibili
Partecipante	-	Membro di un Team che partecipa all' Hackathon
Giudice	-	Utente selezionato che può giudicare e valutare gli elaborati delle squadre
Organizzatore	-	Definisce le informazioni necessarie per pubblicare un Hackathon
Team	- Nome - NumeroMembri	Squadra di Partecipanti che ha il compito di pubblicare i progressi durante la competizione
Documento	- Titolo - Contenuto - Commento	File prodotto dai vari Team che verrà poi valutato dai Giudici
Hackathon	- Titolo - Sede - Durata - Datainizio - DataFine - PeriodoIscrizioni - DataAperturaIscrizioni - DataChiusuraIscrizioni - MaxIscritti - MaxDimTeam - DescrizioneProblema	Competizione istituita da un Organizzatore

Tabella 1: Descrizione delle entità del sistema

2.5 Dizionario delle Relazioni

Relazioni	Tipologia	Descrizione
Competizione	Uno-a-Molti	- Un Team compete in un 1 singolo hackathon - In 1 Hackathon competono nessuno o più Team
Pubblicazione	Uno-a-Molti	- Un team pubblica nessuno o più Documenti - Un Documento è pubblicato da 1 ed un solo Team
Esaminazione	Molti-a-Molti	- Un Giudice esamina nessuno o più Documenti - Un Documento è esaminato da nessuno o più Giudici
Organizzazione	Uno-a-Molti	- Un Organizzatore organizza 1 o più Hackathon - un hackathon è organizzato da 1 Organizzatore
Selezione	Molti-a-Molti-a-Molti	- Un Organizzatore seleziona 2 o più Giudici nel contesto di 1 o più Hackathon - Un Giudice è selezionato da 1 o più Organizzatori nel contesto di 1 o più Hackathon - In un Hackathon sono selezionati 2 o più Giudici da 1 Organizzatore
Partecipazione	Molti-a-Molti	- Un Partecipante partecipa a 1 o più Team - Un Team possiede 1 più Partecipanti
Registrazione	Molti-a-Molti	- Data: attributo di relazione per memorizzare la data di Registrazione all' hackathon - Un Partecipante si registra a 1 o più Hackathon - Ad un Hackathon si Registrano 1 o più Partecipanti
Votazione	Molti-a-Molti	- Voto: attributo di relazione che individua il valore numerico della votazione - Un Team è valutato da nessuno o più Giudici - Un Giudice valuta nessuno o più Team

Tabella 2: Dizionario delle relazioni tra le entità individuate

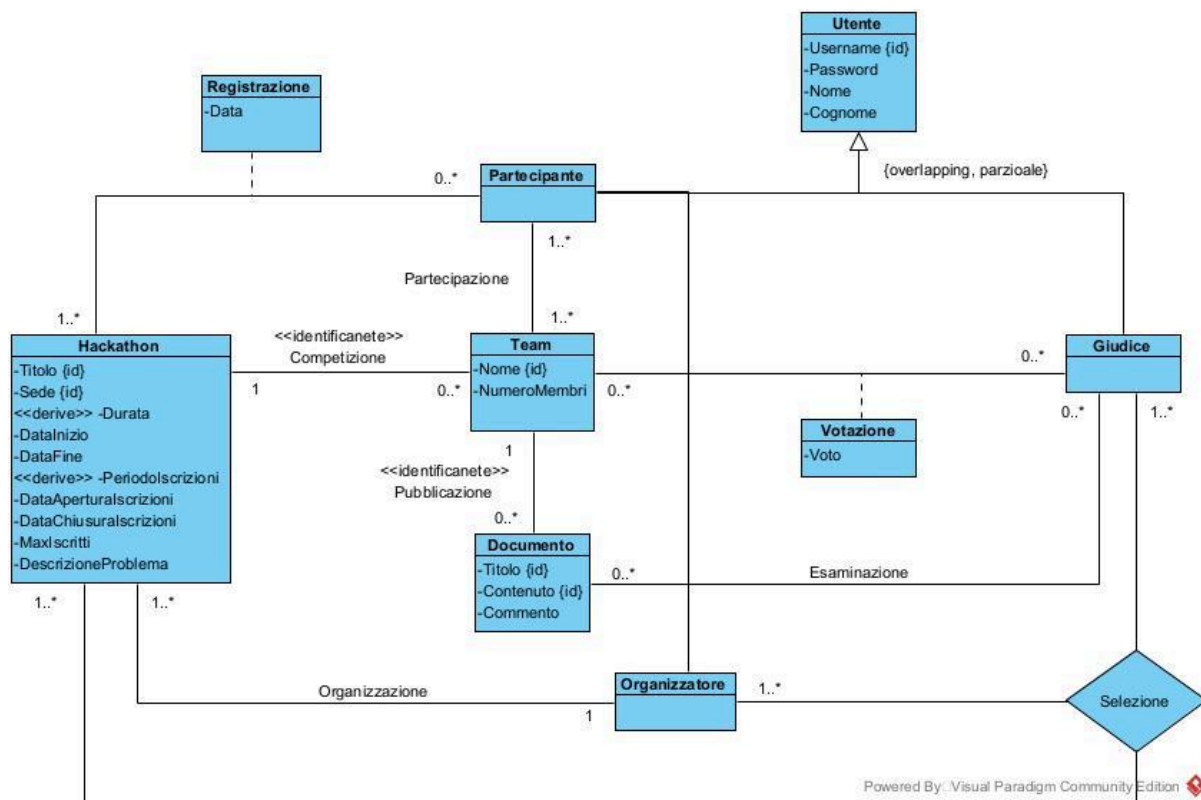
2.6 Diagramma Concettuale - ER

Definizione: Tale Diagramma Entity–Relationship (ER) è un modello concettuale utilizzato per rappresentare in forma grafica le entità individuate di dominio, i loro attributi e le relazioni che intercorrono tra di esse.

Questo nella presente fase di progettazione concettuale, consente di descrivere la struttura dei dati in modo indipendente dall'implementazione fisica che avverrà successivamente.

2.7 Diagramma Concettuale – UML

Tale versione del modello concettuale è espresso nei termini e formalismi descritti dall'**Unified Modeling Language**.



2.8 Ristrutturazione del Modello Concettuale

Introduzione

Definizione: La determinazione del modello concettuale precedentemente illustrata deve essere seguita da un processo di traduzione tale da portare a stabilire uno **schema logico**. Quest'ultimo processo è definito *ristrutturazione del modello concettuale*, una fase il cui obiettivo è quello di definire uno schema logico in grado di descrivere tutte le informazioni contenute nello schema concettuale prodotto dalla fase di progettazione concettuale.

Un Modello logico infatti, presenta delle restrizioni rispetto al modello concettuale in quanto agli elementi rappresentabili in esso.

Degli elementi costituenti un modello concettuale i seguenti non sono rappresentabili nel logico:

- **Gerarchie**
- **Attributi Multipli**
- **Attributi Strutturali**

Tale fase di ristrutturazione si basa su:

- Criteri di ottimizzazione dello schema
- Regole di traduzione verso il modello logico

A partire dal **diagramma concettuale** e dal **carico applicativo** eventuale, si ottiene attraverso i seguenti step , il **diagramma ristrutturato**.

Fasi del percorso di ristrutturazione

Il processo di ristrutturazione si articola nelle seguenti fasi operative:

1. Analisi delle Ridondanze
2. Eliminazione delle Generalizzazioni
3. Eliminazione degli Attributi Multivalore
4. Eliminazione degli Attributi Strutturali
5. Partizionamento/Accorpamento di Entità e Associazioni
6. Scelta degli Identificatori principali

2.8.1 Analisi delle Ridondanze

Definizione: Una ridondanza in uno schema concettuale corrisponde alla presenza di un dato duplicato e/o derivabile da altri dati.

Tale fase di analisi si prefigge l'obiettivo di individuare informazioni ritenibili tali e successivamente applicare una delle seguenti tecniche di risoluzione:

- **Mantenimento della ridondanza:** La ridondanze individuate vengono mantenute per motivi di efficienza (es. campi frequentemente consultati).
- **Eliminazione della ridondanza:** Le ridondanze individuate vengono rimosse poiché potrebbero causare errori e/o aumentare costi di aggiornamento

Dato derivabile

Definizione: Un dato derivabile è un'informazione che può essere ottenuta attraverso operazioni di calcolo, elaborazione o interrogazione su altri dati già presenti nella base di dati. La sua memorizzazione esplicita rappresenta una forma di ridondanza.

Individuazione DATi Derivabili:

Nel contesto di una progettazione concettuale, è possibile che

1. Attributi e
2. Relazioni

vengano ritenuti possibilmente derivabili.

Attributo Derivabile

Definizione: Un attributo derivabile è un attributo il cui valore può essere calcolato o determinato a partire dai valori di altri attributi della stessa entità o di entità correlate attraverso operazioni quali:

- Operazioni aritmetiche (somma, differenza, prodotto, divisione)
- Funzioni di aggregazione (successivamente illustrate)
- Operazioni temporali (differenze tra date, calcoli di età)

Individuazione Attributi Derivabili

Gli attributi di dominio ritenuti **derivabili** individuati sono i seguenti:

1. **NumeroMembri**
2. **PeriodoIscrizioni**
3. **Durata**

Numeromembri Durante la progettazione concettuale è stata presa in considerazione la possibilità di includere nell'elenco degli attributi dell'entità Team, l'attributo NumeroMembri, che sta ad indicare il numero di membri componenti un determinato Team. Quest'ultimo, nella fase di ristrutturazione è stato classificato come derivabile in quanto sarebbe necessario effettuare il solo conteggio dei membri di un Team. Potrebbe essere una potenziale ridondanza.	Periodoliscrizioni In secondo luogo, Periodoliscrizioni è un attributo specificato dal dominio e rappresenta l'intervallo di tempo che va dall'apertura delle iscrizioni dell hackathon fino alla loro chiusura. E ` stato ritenuto quindi anch'esso derivabile, in quanto calcolabile tramite la differenza dei due attributi sopra elencati. Anch'esso, quindi, potrebbe essere una potenziale ridondanza.	Durata Anche Durata è un attributo specificato dalla richiesta del cliente e descrive la durata in giorni dell'intera competizione. Come nel caso precedente quindi, sarebbe possibile derivare questo attributo dalla differenza tra le due date di inizio e fine gara. ` E quindi di fatto una potenziale ridondanza.
--	---	--

Per determinare se questi attributi costituiscono:

- Una **ridondanza effettiva oppure**
- rappresentino **un'informazione rilevante da mantenere**

è stato valutato ogni attributo con i seguenti risultati:

1. Attributo NumeroMembri (Entità: Team)

- a. (Ipotesi): Si ipotizzano 100 team nel database, con una media di 15 membri per team. Le operazioni previste sono 50 nuovi iscritti al giorno e 200 visualizzazioni dei team al giorno.
- b. Confronto accessi giornalieri: 450 (se lo memorizziamo) contro 3350 (se lo calcoliamo ogni volta).
- c. Costo in memoria: 400 byte.
- d. Si è deciso di **mantenere** questo attributo.
Calcolare ogni volta i membri di un team richiede troppi accessi al database per un'operazione così frequente. Memorizzare il dato garantisce un risparmio enorme (-86,6%) a costo zero per la memoria.

2. Attributo PeriodoIscrizioni (Entità: Hackathon)

- a. (Ipotesi): Si ipotizzano 50 hackathon nel database, visualizzati circa 150 volte al giorno.
- b. Confronto accessi giornalieri: 150 (se lo memorizziamo) contro 300 (se lo calcoliamo ogni volta).
- c. Costo in memoria: 200 byte.
- d. Si è deciso di **scartare** questo attributo.
Il risparmio di accessi è troppo basso per giustificare la ridondanza. Il calcolo (una banale differenza tra due date) può essere fatto dal software senza pesare sul database, evitando così anche il rischio di avere dati non aggiornati o contraddittori.

3. Attributo Durata (Entità: Hackathon)

- a. (Ipotesi): Sempre 50 hackathon nel sistema. Si stimano 5 creazioni di eventi al giorno, 20 ricerche specifiche per durata e 5 visualizzazioni di statistiche globali.
- b. Confronto accessi giornalieri: 460 (se lo memorizziamo) contro 2500 (se lo calcoliamo ogni volta).
- c. Costo in memoria: 200 byte.
- d. Si è deciso di **mantenere** questo attributo.
Senza questo attributo salvato, ogni volta che un utente fa una ricerca o guarda le statistiche, il database è costretto a leggere tutti i 50 hackathon e calcolarne la durata al volo. Memorizzarlo ottimizza pesantemente le prestazioni e le ricerche.

Relazione Derivabile

Definizione: Una relazione derivabile è una relazione che può essere ricostruita attraverso la composizione di altre relazioni esistenti nel database mediante operazioni di:

- Join multipli attraverso percorsi di relazioni intermedie
- Operazioni insiemistiche ($\cup$, $\cap$, $\setminus$)
- Proiezioni e selezioni su relazioni esistenti (σ , π)- Aggregazioni su strutture relazionali complesse

Individuazione Relazioni Derivabili

La relazione presente nella progettazione ritenuta derivabile è la seguente:

- Registrazione

Registrazione
Hackathon -----> Partecipante

La relazione in questione presenta una potenziale criticità che richiede un'attenta valutazione progettuale. Dal punto di vista della teoria relazionale, questa relazione potrebbe essere considerata ridondante poiché derivabile attraverso la composizione delle relazioni già esistenti nel modello dati.

Formalmente, la relazione **Registrazione** potrebbe essere espressa ugualmente tramite il seguente percorso alternativo:

Competizione Partecipazione
Hackathon -----> Team -----> Partecipante

Tutto ciò solleva interrogativi fondamentali sulla necessità di mantenere esplicitamente la relazione **Registrazione**. L'eliminazione di tale ricorsione consentirebbe di:

- Ridurre lo spazio di memorizzazione occupato
- Eliminare potenziali inconsistenze dovute alla ridondanza
- Semplificare la struttura del modello dati

Tuttavia, questa valutazione puramente teorica potrebbe non considerare adeguatamente gli impatti operativi che deriverebbero dall'adozione di un approccio basato esclusivamente sulla derivazione delle informazioni registrate.

La questione centrale da risolvere riguarda quindi determinare se la relazione **Registrazione** costituisca effettivamente una ridondanza dannosa da eliminare oppure se rappresenti una scelta giustificata.

1. Relazione Derivabile Registrazione (Hackathon - Partecipante)

- a. (Ipotesi): Si ipotizzano 1500 registrazioni nel database (30 partecipanti per 50 hackathon). Le operazioni previste ogni giorno sono: 200 nuove registrazioni, 50 visualizzazioni della lista partecipanti e 100 verifiche per controllare se un utente è abilitato a formare un team.
- b. Confronto accessi giornalieri: 3900 (se salviamo la relazione direttamente) contro 7400 (se la calcoliamo in modo indiretto passando per i team a cui i partecipanti appartengono).
- c. Costo in memoria: 18 KB (calcolato su 12 byte per record).

- d. Si è deciso di **mantenere** questa relazione.
Il risparmio di accessi è notevole (-47,3%). Soprattutto, offre vantaggi architetturali fondamentali: permette di registrare un partecipante a un hackathon prima e indipendentemente dal fatto che sia già stato assegnato a un team, garantendo una migliore gestione degli stati e un'ottima scalabilità.

2.8.2 Eliminazione delle Generalizzazioni

Poiché i sistemi tradizionali per la gestione di basi di dati non consentono di rappresentare direttamente una generalizzazione, è necessario tradurle usando altri costrutti della modellazione concettuale. Le opzioni possibili sono le seguenti:

1. Accorpamento delle figlie della generalizzazione nell'entità padre

Questa tecnica consente nel **trasportare** attributi e relative associazioni presenti nelle entità figlie all'interno dell'entità padre. Gli eventuali attributi devono poter presentare **valore nullo** e le eventuali relazioni devono esibire un **vincolo di partecipazione parziale**, per poter rappresentare appieno tutte le entità soppresse.

2. Accorpamento del padre della generalizzazione nelle entità figlie

La seconda tecnica è *il diretto opposto della prima*, nella quale si procede **inglobando** l'entità generale nelle sue specializzazioni. I suoi attributi si distribuiscono tra le figlie e le sue associazioni si moltiplicano per collegarsi alle entità restanti, preservando gli stessi **vincoli di cardinalità**.

3. Sostituzione della generalizzazione con associazioni

Nell'ultima tecnica si procede invece preservando tutte le entità coinvolte e **introducendo delle associazioni** tra il padre e ogni sua figlia, conservando quelle già presenti. Queste nuove associazioni — dette is-a — sono di tipo **1:1 con vincolo di partecipazione parziale verso il padre**; questo rappresenta correttamente l'essenza di una generalizzazione, dove ogni figlia è necessariamente in relazione col padre ma non viceversa.

E' bene sottolineare che le entità specializzate andrebbero in questa fase descritte come **deboli** e identificate tramite le relazioni appena create, ma il successivo processo di identificazione di chiavi principali consente di sorvolare questo dettaglio.

Una volta descritte le diverse possibilità di ristrutturazione della gerarchia, si illustrano adesso i **motivi dietro alla scelta** di una di esse come segue:

Alternativa 1	Alternativa 2	Alternativa 3
Questa opzione è da evitare in quanto, assegnando tutte le operazioni dei vari utenti — che differiscono significativamente — ad una <i>singola entità</i> , introdurremo un ammontare di valori nulli eccessiva, oltre a complicare in modo non banale la chiarezza del sistema.	Questa tecnica è invece inattuabile poiché la generalizzazione presenta un <i>vincolo di completezza parziale</i> ; ciò significa che accorpare Utente nelle entità figlie porterebbe ad una perdita di rappresentazione per quelle istanze di Utente che non cadono in nessuna delle tre specializzazioni.	La soluzione ritenuta pertinente è stata quindi la presente in quanto garantisce assenza di valori nulli nelle entità e la corretta rappresentazione del mini-world, al costo di un numero di accessi aggiuntivi che non impatta gravosamente sulle prestazioni della base di dati.

2.8.3 Eliminazione Attributi Multivalore

Durante la progettazione concettuale non sono stati individuati attributi multivalore da elidere eventualmente, pertanto si procede con la fase successiva.

2.8.4 Eliminazione Attributi Strutturali

Poiché non sono presenti attributi strutturati da scomporre, è stato quindi possibile procedere con l'ultima fase della ristrutturazione.

2.8.5 Partizione/Accorpamento di Entità e Relazioni

Le entità e associazioni individuate non necessitano di essere modificate ulteriormente tramite partizionamento o accorpamento, segue quindi l'analisi dell'ultima fase della ristrutturazione del modello concettuale.

2.8.6 Scelta degli Identificatori Principali

In questa sezione si illustra la scelta delle chiavi candidate per la successiva introduzione delle chiavi primarie del sistema:

- **Utente:** l'attributo *Username* è stato ritenuto adatto per essere un attributo identificatore poiché tramite quest'ultimo è possibile distinguere univocamente ogni utente.
- **Partecipante — Organizzatore — Giudice:** in quanto specializzazioni dell'entità Utente, faranno riferimento all'attributo identificatore *Username* della generalizzazione **Utente**. Per rendere più forte il vincolo di unicità, introduciamo quindi un attributo **ID** per Partecipante, Organizzatore e Giudice per identificare la precisa istanza specializzata, permettendo inoltre di preservare la semplicità nel sistema.
- **Team:** Durante la presente fase di scelta degli identificatori, un attributo di dominio già presente ha rappresentato un eventuale candidato:
 - a. l'attributo *Nome* avrebbe costituito un'opzione valida in quanto nel contesto di singolo Hackathon non possono esistere due Team con lo stesso nome. D'altra parte considerando la base di dati nel suo insieme tale attributo non impone un vincolo di unicità tanto forte da poter identificare un singolo Team.

Quindi è opportuno introdurre un nuovo attributo identificatore "idTeam".

- **Documento** L'entità Documento non presenta attributi tali da poter costruire un identificatore per via dello stretto legame con **Team**: pertanto è stato necessario introdurre un attributo che potesse invece rivestire quest'ultimo ruolo adeguatamente: "idDocumento".
- Nell'entità **Hackathon**, l'identificazione univoca delle istanze presenta una problematica progettuale che richiede un'analisi formale delle alternative disponibili: Dal punto di vista teorico, l'identificazione naturale di un hackathon può essere espressa attraverso la chiave candidata composta:

(Titolo, Sede)

Questa scelta si basa sulla considerazione secondo cui l'attributo *Titolo* da solo non garantisce l'unicità, poiché nella presente progettazione concettuale è stato considerato opportuno prevedere scenari in cui hackathon con denominazioni identiche possano svolgersi in contesti geografici diversi.

La combinazione *Titolo x Sede* fornisce invece un identificatore semanticamente significativo ed univoco.

Tuttavia, l'adozione di una chiave primaria composta introduce significative **complessità operative** nell'implementazione del sistema:

- **Propaganda referenziale:** ogni relazione che referencia *Hackathon* deve trasportare entrambi gli attributi come chiave esterna composta. E' stata quindi adottata una soluzione di compromesso basata sull'introduzione di una chiave surrogata "idHackathon" mantenendo contemporaneamente il vincolo di unicità sulla chiave naturale:

UNIQUE(Titolo, Sede)

Facendo in questo modo è preservata la semantica del dominio attraverso il vincolo di unicità naturale , mentre la chiave surrogata garantisce:

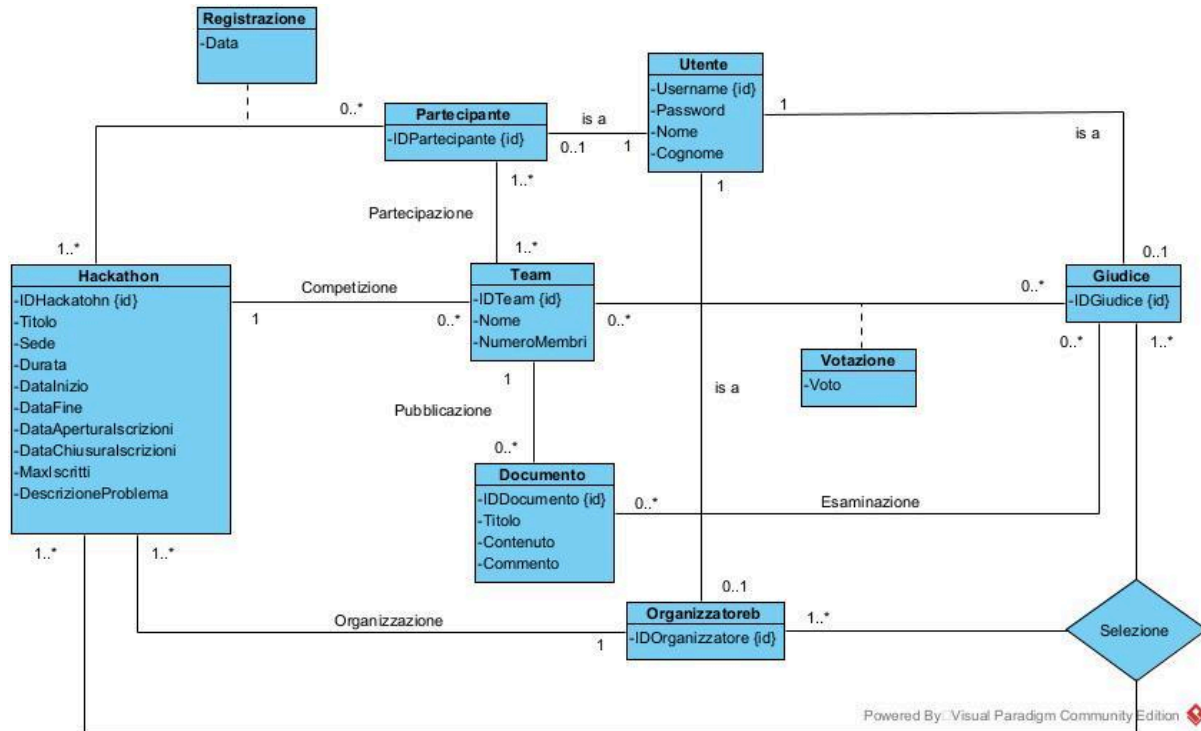
- Semplicità nelle relazioni referenziali (singolo attributo intero).

2.8.7 Diagramma concettuale Ristrutturato — ER

Definizione: Un **diagramma ristrutturato** è il risultato della sottoposizione del modello concettuale al processo di ristrutturazione. Esso mantiene invariato il contenuto informativo dello schema originale ma pone le basi strutturali per la successiva traduzione dello stesso nello schema logico.

2.8.8 Diagramma concettuale Ristrutturato — UML

Diagramma ristrutturato espresso tramite i formalismi dell'UML



2.8.9 Dizionario delle Entità

Entità	Attributi	Descrizione
Utente	- <u>Username</u> - Password - Nome - Cognome	Persona Iscritta Alla piattaforma che può prendere parte agli eventi disponibili
Partecipante	- <u>idPartecipante</u>	Membro del Team che partecipa alla Hackathon
Giudice	- <u>idGiudice</u>	Utente Selezionato Che Può giudicare valutare gli elaborati delle squadre
Organizzatore	- <u>idOrganizzatore</u>	Definisce Le Informazioni necessarie per pubblicare un Hackathon
Team	- <u>idTeam</u> - Nome - NumeroMembri	Squadra Partecipanti che ha il compito di pubblicare progressi durante la competizione
Documento	- <u>idDocumento</u> - Titolo - Contenuto - Commento	File prodotto dai vari Team che verrà poi valutato dai Giudici
Hackathon	- <u>idHackathon</u> - Titolo - Sede - Durata - Datainizio - DataFine - DataAperturalscrizioni - DataChiusuralscrizioni - MaxPartecipanti - DimMaxTeam - DescrizioneProblema	Competizione Istituita Da Un Organizzatore

Tabella 3: Descrizione delle entità dopo la **ristrutturazione**

2.8.10 Dizionario delle Relazioni

Relazioni	Tipologia	Descrizione
Competizione	Uno-a-Molti	- Un Team compete in un singolo Hackathon - In 1 Hackathon competono nessuno o più Team
Pubblicazione	Uno-a-Molti	- Un Team pubblica nessuno o più Documenti - Un Documento è pubblicato da 1 ed un solo Team
Esaminazione	Molti-a-Molti	- Un Giudice esamina nessuno o più Documenti - Un Documento è esaminato da nessuno o più Giudici
Organizzazione	Uno-a-Molti	- Un Organizzatore organizza 1 o più Hackathon - Un Hackathon è organizzato da 1 Organizzatore
Selezione	Molti-a-Molti-a-Molti	- Un Organizzatore seleziona 2 o più Giudici nel contesto di 1 o più Hackathon - Un Giudice è selezionato da 1 o più Organizzatori nel contesto 1 o più Hackathon - In un Hackathon sono selezionati 2 o più Giudici da 1 Organizzatore
Partecipazione	Molti-a-Molti	- Un Partecipante partecipa ad 1 o più Team - Un Team possiede 1 o più Partecipanti

Registrazione	Molti-a-Molti	- Data: attributo di relazione per memorizzare la data di Registrazione all'Hackathon - Un Partecipante si registra a 1 o più Hackathon - Ad un Hackathon si registrano nessuno o più Partecipanti
Votazione	Molti-a-Molti	- Voto: attributo di relazione che individua valore numerico della votazione - Un Team è valutato da nessuno o più Giudici - Un Giudice valuta nessuno o più Team
Is a (Partecipante)	Uno-a-Molti	Un Utente può essere un Partecipante; Un Partecipante è un Utente
Is a (Organizzatore)	Uno-a-Molti	Un Utente pu`essere un Organizzatore; Un Organizzatore è un Utente
Is a (Giudice)	Uno-a-Molti	Un Utente può essere un Giudice; Un Giudice è un Utente

Tabella 4: Descrizione delle relazioni tra le entità individuate

2.8.11 Dizionario dei Vincoli

#	Nome Vincolo	Tipologia	Descrizione
1	Durata positiva	Intra-Relazionale	La durata Hackathon deve essere strettamente maggiore di 0.
2	Date corrette	Intra-Relazionale	Le date di apertura, chiusura iscrizioni, inizio e fine devono rispettare l'ordine logico: apertura < chiusura iscrizioni < inizio < fine
3	Limite inferiore per <i>MaxIscritti</i>	Intra-Relazionale	Un Hackathon deve consentire l'iscrizione ad almeno 2 Partecipanti .
4	Limite inferiore per <i>MaxDimTeam</i>	Intra-Relazionale	Ogni Team deve consentire la presenza un numero di membri > di 0
5	Vincolo di unicità per la coppia (<i>Titolo, sede</i>)	Intra-Relazionale	La coppia di attributi (<i>Titolo, Sede</i>) è unica per ogni Hackathon .
6	membri di un Team positivi	Intra-Relazionale	Il Numero Di Membri in un Team deve essere > 0.
7	Dominio del voto	Intra-Relazionale	Il Voto Finale Deve Essere Compreso tra 0 e 10 estremi inclusi
8	Vincolo di disgiunzione intra-hackathon	Inter-Relazionale	Nel contesto di un singolo Hackathon , un Partecipante non può ricoprire un secondo ruolo.
9	Team minimi per Hackathon	Inter-Relazionale	Un Hackathon, per essere considerato valido, deve prevedere almeno 2 Team.
10	Giudici Minimi	Inter-Relazionale	Un gruppo di Giudici per un Hackathon Deve Essere Composto almeno 2 membri.
11	Vincolo di unicità per l'attributo <i>Nome</i>	Inter-Relazionale	L'attributo Nome di un Team deve essere univoco per ogni Hackathon .
12	Completezza di esame	Inter-Relazionale	Prima Di inserire una Votazione per un Team , un Giudice deve aver

			esaminato prima tutti i relativi documenti
13	Vincolo di unicità per gli attributi <i>titolo</i> e <i>Contenuto</i>	Inter-Relazionale	Non devono esistere duplicazioni degli attributi <i>Titolo</i> e <i>Contenuto</i> di un documento contemporaneamente nel contesto del hackathon.
14	Correttezza dell'iscrizione	Inter-Relazionale	Un Partecipante può iscriversi ad un Hackathon solo nella finestra temporale disponibile
15	Correttezza dell'inizio competizione	Inter-Relazionale	Un Hackathon non può essere avviato in date diverse da quelle indicate nel relativo record.
16	Esistenza di un progresso	Inter-Relazionale	Un Team deve aver pubblicato almeno un progresso nella competizione di riferimento prima di poter ricevere una votazione.
17	Correttezza del cambio team	Inter-Relazionale	Per poter cambiare Team , un Partecipante deve rispettare i vincoli temporali e i limiti di dimensione imposti dall'Organizzatore .
18	Completezza delle votazioni	Inter-Relazionale	Prima di poter produrre la tabella dei vincitori, è necessario che ogni Team Presenti tutti i voti necessari.

Tabella 6: Vincoli del Sistema

3 Progettazione Logica

Introduzione

Definizione: La progettazione logica è il processo di trasformazione dello schema concettuale in uno **schema logico** che rappresenta la struttura dei dati secondo un particolare modello logico dei dati.

Per la presente piattaforma il modello logico scelto è stato il modello relazionale. Nel contesto di utilizzo di tale la progettazione logica comporta:

- **Per ogni entità nel modello concettuale:** uno schema di relazione con lo stesso nome avente
 - a. i medesimi attributi dell' entità
 - b. per chiave il suo identificatore

- **Per ogni relazione nel modello concettuale:** uno schema di relazione con lo stesso nome avente
 - a. per attributi: gli attributi della relazione
 - b. per chiave: gli identificatori delle entità coinvolte

Tutto ciò distinguendo i vari casi vase ai vincoli di partecipazione.

E' quindi necessario un processo di traduzione sistematica tale che, a partire dalle entità e dalle relazioni individuate nel modello concettuale, permetta di definire formalmente i corrispondenti nel modello logico relazionale. IL suddetto processo è quello del **mapping logico-relazionale**.

3.1 Mapping Logico-Relazionale

Di seguito vengono dettagliate le regole formali di traduzione applicate a ciascuna entità e relazione del modello concettuale:

3.1.1 Entità

Utente Mappata nella relazione

Utente (username, password, nome, cognome)

Giudice Mappata nella relazione

Giudice (id_giudice)

Documento Mappata nella relazione

Documento (id_documento, commento, contenuto, titolo)

Team Mappata nella relazione

Team (id_team, nome, numero_membri)

Partecipante Mappato nella relazione

Partecipante (id_partecipante)

Hackathon Mappato nella relazione

Hackathon (id_hackathon, titolo, sede, durata, data_inizio, data_fine,
descrizione_problema, dat_apertura_iscrizioni, data_chiusura_iscrizioni, max_iscritti,
max_dim_team)

Organizzazione Mappata nella relazione

Organizzazione (id_organizzazione)

3.1.2 Relazione Multi-a-Multi-a-Multi (M:N:P)

Relazione Selezione

Tabelle coinvolte:

- Giudice
- Organizzatore
- Hackathon

Mapping: Viene introdotta una nuova relazione:

Selezione (id_giudice, id_organizzazione, id_hackathon)

provvista degli attributi chiave delle tabelle coinvolte.

3.1.3 Relazioni Molti-a-Molti (N:N)

Relazione *Esaminazione*

Tabelle coinvolte:

- Giudice
- Documento

Mapping: Viene indotta una nuova relazione:

Esaminazione (id_giudice, id_documento)

provvista degli attributi chiave primaria delle entità coinvolte.

Relazione *Votazione*

Tabelle coinvolte:

- Team
- Giudice

Mapping: Viene indotta una nuova relazione:

Votazione (id_giudice, id_team, voto)

provvista degli attributi chiave primaria delle entità coinvolte e dell'attributo descrittivo **voto**.

Relazione *Partecipazione*

Tabelle coinvolte:

- Partecipante
- Team

Mapping: Viene indotta una nuova relazione:

Partecipazione(id_partecipante, id_team)

provvista degli attributi chiave primaria delle entità coinvolte.

Relazione *Registrazione*

Tabelle coinvolte:

- Partecipante
- Hackathon

Mapping: Viene indotta una nuova relazione:

Registrazione (id_partecipante, id_hackathon, data)

provvista degli attributi chiave primaria delle entità coinvolte e dell'attributo accessorio **data**.

3.1.4 Relazioni Uno-a-Molti (1:N)

Relazione *Organizzazione*

tabelle coinvolte:

- Organizzatore
- Hackathon

Mapping: La chiave primaria dell'entità **Organizzatore** viene inserita come chiave esterna della relazione **Hackathon**

Relazione *Pubblicazione*

Tabelle coinvolte:

- Team
- Documento

Mapping: La chiave primaria dell'entità **Team** viene inserita come chiave esterna nella relazione **Documento**

Relazione *Competizione*

Tabelle coinvolte:

- Hackathon
- team

Mapping: La chiave primaria dell'entità **Hackathon** viene inserita come chiave esterna nella relazione **Team**

3.1.5 Relazioni uno-a-uno (1:1)

Relazione *Is a* – *Partecipante*

Tabelle coinvolte:

- Utente
- Partecipante

Mapping: La chiave primaria dell'entità **Utente** viene inserita come chiave esterna nella relazione **Partecipante**

Relazione *Is a* – *Organizzatore*

Tabelle coinvolte:

- Utente
- Organizzatore

Mapping: La chiave primaria dell'entità **Utente** viene inserita come chiave esterna nella relazione **Organizzatore**

Relazione *Is a* – *Giudice*

Tabelle coinvolte:

- Utente
- Giudice

Mapping: La chiave primaria dell'entità **Utente** viene inserita come chiave esterna nella relazione **Giudice**

3.1.6 Schema Logico-Relazionale

Il seguente schema logico-relazionale rappresenta il **risultato** del processo di **mapping logico-relazionale** applicato agli elementi costituenti lo schema concettuale ristrutturato precedentemente illustrato.

Utente (username, password, nome, cognome)

Partecipante (id_partecipante, username)

Partecipante.username → Utente.username

Organizzazione (id_organizzazione, username)

Organizzatore.username → Utente.username

Giudice (id_giudice, username)

Giudice.username → Utente.username

Esaminazione (id_giudice, id_documento)

Esaminazione.id_giudice → Giudice.id_giudice

Esaminazione.id_documento → Documento.id_documento

Documento (id_documento, commento, contenuto, titolo, id_team)

Documento.id_team → Team.id_team

Team (id_team, nome, numero_membri, id_hackathon)

Tea.id_hackathon → Hackathon.id_hackathon

Partecipazione(id_partecipante, id_team)

Partecipazione.id_partecipante → Partecipante.id_partecipante

Partecipazione.id_team → Team.id_team

Hackathon (id_hackathon, titolo, sede, durata, data_inizio, data_fine, descrizione_problema, dat_apertura_iscrizioni, data_chiusura_iscrizioni, max_iscritti, max._dim_team, id_organizzatore)

Hackathon.id_organizzatore → Organizzatore.id_organizzatore

Votazione (id_giudice, id_team, voto)

Votazione.id_team → Team.id_team

Votazione.id_giudice → Giudice.id_giudice

Selezione (id_giudice, id_organizzazione, id_hackathon)

Selezione.id_giudice → Giudice.id_giudice

Selezione.id_organizzatore → Organizzatore.id_organizzatore

Selezione.id_hackathon → Hackathon.id_hackathon

Registrazione (id_partecipante, id_hackathon, data)

Registrazione.id_partecipante → Partecipante.id_partecipante

Registrazione.id_hackathon → Hackathon.id_hackathon

per convenzione abbiamo usato: Attributo → Chiave Primaria della Relazione
Attributo → Chiave Esterna

4 Progettazione Fisica

Introduzione

Definizione: La progettazione fisica rappresenta la fase del processo di progettazione in cui si definiscono gli aspetti implementativi riguardo la memorizzazione effettiva dei dati e la gestione di questi da parte del DBMS.

Definizione dello Schema Fisico

4.1 Definizione delle Tabelle Fisiche

La seguente implementazione in linguaggio SQL rispetta l'ordine di creazione delle tabelle necessario per garantire l'integrità referenziale delle chiavi esterne.

```
-- 1. Tabella base senza dipendenze
CREATE TABLE IF NOT EXISTS Utente (
  username VARCHAR (30) PRIMARY KEY,
  nome VARCHAR (30) NOT NULL,
  cognome VARCHAR (30) NOT NULL,
  password VARCHAR (100) NOT NULL
);
```

```
-- 2. Tabelle che dipendono da Utente
CREATE TABLE IF NOT EXISTS Organizzatore (
  id_organizzatore SERIAL PRIMARY KEY,
  username VARCHAR (30) NOT NULL,
  FOREIGN KEY (username) REFERENCES Utente(username)
  ON UPDATE CASCADE,
  ON DELETE CASCADE
);
```

```
CREATE TABLE IF NOT EXISTS Partecipante (
  id_partecipante SERIAL PRIMARY KEY,
  username VARCHAR (30) NOT NULL,
  FOREIGN KEY (username) REFERENCES Utente(username)
  ON UPDATE CASCADE,
  ON DELETE CASCADE
);
```

```

CREATE TABLE IF NOT EXISTS Giudice (
  id_giudice SERIAL PRIMARY KEY,
  username VARCHAR (30) NOT NULL,
  FOREIGN KEY (username) REFERENCES Utente(username)
  ON UPDATE CASCADE,
  ON DELETE CASCADE
);

```

– 3. Tabelle che dipendono da Organizzatore

```

CREATE TABLE IF NOT EXISTS Hackathon (
  id_hackathon SERIAL PRIMARY KEY,
  titolo VARCHAR(50) NOT NULL,
  sede VARCHAR(30) NOT NULL,
  durata INT NOT NULL,
  data_inizio DATE NOT NULL,
  data_fine DATE NOT NULL,
  descrizione_problema TEXT DEFAULT 'Descrizione assente.',
  data_apertura_iscrizioni DATE NOT NULL,
  data_chiusura_iscrizioni DATE NOT NULL,
  max_iscritti INT NOT NULL,
  max_dim_team INT NOT NULL,
  id_organizzatore INT NOT NULL DEFAULT 0,
  FOREIGN KEY (id_organizzatore) REFERENCES Organizzatore(id_organizzatore)
  ON UPDATE CASCADE,
  ON DELETE SET DEFAULT,
  CONSTRAINT durata_positiva CHECK (durata > 0), /* [1] */
  CONSTRAINT date_corrette CHECK ( /* [2] */
    (data_apertura_iscrizioni < data_chiusura_iscrizioni)
    AND (data_chiusura_iscrizioni <= data_inizio)
    AND (data_inizio < data_fine) ),
  CONSTRAINT max_iscritti_minimo CHECK (max_iscritti > 1), /* [3] */
  CONSTRAINT max_dim_team_minima CHECK (max_dim_team > 0), /* [4] */
  CONSTRAINT chiave_naturale UNIQUE (titolo, sede) /* [5] */
);

```

-- 4. Tabelle che dipendono da Hackathon

```

CREATE TABLE IF NOT EXISTS Team (
  id_team SERIAL PRIMARY KEY,
  nome VARCHAR(20) NOT NULL,
  numero_membri INT NOT NULL,
  id_hackathon INT NOT NULL,
  FOREIGN KEY (id_hackathon) REFERENCES Hackathon(id_hackathon)
  ON UPDATE CASCADE
  ON DELETE CASCADE,
  CONSTRAINT membri_positivi CHECK (numero_membri > 0) /* [6] */
);

```

-- 5. Tabelle che dipendono da Team

```

CREATE TABLE IF NOT EXISTS Documento (

```

```

id_documento SERIAL PRIMARY KEY,
commento TEXT DEFAULT 'Commento assente.',
contenuto TEXT NOT NULL,
titolo VARCHAR(50) NOT NULL,
id_team INT NOT NULL,
FOREIGN KEY (id_team) REFERENCES Team(id_team)
ON UPDATE CASCADE
ON DELETE CASCADE
);

```

```

-- 7. Tabelle di Relazione
CREATE TABLE IF NOT EXISTS Registrazione (
  id_partecipante INT NOT NULL DEFAULT 0,
  id_hackathon INT NOT NULL,
  data DATE NOT NULL,
  PRIMARY KEY (id_partecipante, id_hackathon),
  FOREIGN KEY (id_partecipante) REFERENCES Partecipante(id_partecipante)
  ON UPDATE CASCADE
  ON DELETE SET DEFAULT,
  FOREIGN KEY (id_hackathon) REFERENCES Hackathon(id_hackathon)
  ON UPDATE CASCADE
  ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS Partecipazione (
  id_team INT NOT NULL,
  id_partecipante INT NOT NULL DEFAULT 0,
  PRIMARY KEY (id_team, id_partecipante),
  FOREIGN KEY (id_team) REFERENCES Team(id_team)
  ON UPDATE CASCADE
  ON DELETE CASCADE,
  FOREIGN KEY (id_partecipante) REFERENCES Partecipante(id_partecipante)
  ON UPDATE CASCADE
  ON DELETE SET DEFAULT
);

CREATE TABLE IF NOT EXISTS Esaminazione (
  id_giudice INT NOT NULL DEFAULT 0,
  id_documento INT NOT NULL,
  PRIMARY KEY (id_giudice, id_documento),
  FOREIGN KEY (id_giudice) REFERENCES Giudice(id_giudice)
  ON UPDATE CASCADE
  ON DELETE SET DEFAULT,
  FOREIGN KEY (id_documento) REFERENCES Documento(id_documento)
  ON UPDATE CASCADE
  ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS Votazione (
  id_giudice INT NOT NULL DEFAULT 0,
  id_team INT NOT NULL,

```

```

voto dominio_voto NOT NULL,
PRIMARY KEY (id_giudice, id_team),
FOREIGN KEY (id_giudice) REFERENCES Giudice(id_giudice)
ON UPDATE CASCADE
ON DELETE SET DEFAULT,
FOREIGN KEY (id_team) REFERENCES Team(id_team)
ON UPDATE CASCADE
ON DELETE CASCADE
);

```

```

-- 8. Tabella di Relazione Ternaria
CREATE TABLE IF NOT EXISTS Selezione (
  id_hackathon INT NOT NULL,
  id_organizzatore INT NOT NULL DEFAULT 0,
  id_giudice INT NOT NULL DEFAULT 0,
  PRIMARY KEY (id_hackathon, id_organizzatore, id_giudice),
  FOREIGN KEY (id_hackathon) REFERENCES Hackathon(id_hackathon)
  ON UPDATE CASCADE
  ON DELETE CASCADE,
  FOREIGN KEY (id_organizzatore) REFERENCES Organizzatore(id_organizzatore)
  ON UPDATE CASCADE
  ON DELETE SET DEFAULT,
  FOREIGN KEY (id_giudice) REFERENCES Giudice(id_giudice)
  ON UPDATE CASCADE
  ON DELETE SET DEFAULT
);

```

La numerazione presente accanto ad ogni vincolo **CONSTRAINT** (es. /* [n] */) corrisponde a quella dei vincoli elencati nel [Dizionario dei Vincoli](#) precedentemente illustrato.

4.2 Implementazione dei Vincoli di Base

Come precedentemente accennato, il *modello relazionale* presenta vari **vincoli formali** che ogni istanza di relazione deve soddisfare per essere correttamente espressa:

1. Vincoli di dominio

Ogni attributi A di una relazione R è associato a un dominio $Dom(A)$, ossia l'insieme dei valori ammessi. Un vincolo sui domini richiede che tutti i valori assunti dagli attributi debbano appartenere al $Dom(A)$.

2. Vincoli di chiave

In una relazione R, ogni attributo di *chiave* deve essere **unico**, senza presentare mai valori duplicati.

3. Vincoli di integrità di entità

Un attributo di *chiave primaria* non può presentare valore NULL per garantire l'esistenza del record.

4. Vincoli di integrità referenziale

Una *chiave esterna* deve sempre riferirsi ad una chiave primaria *esistente* o essere posta a NULL per certificare la **consistenza delle informazioni**.

Questi vincoli formali assumono forma più *esplicita* tramite le seguenti varianti, che descrivono propriamente tramite il linguaggio **SQL** le condizioni necessarie per mantenere **coerenza** nella base di dati:

1. Vincoli sui domini degli attributi
2. Vincoli di chiave primaria
3. Vincoli di integrità referenziale
4. Vincoli sui valori NULL e valori predefiniti
5. Vincoli sulle tuple

1. Vincoli sui domini degli attributi

Un vincolo sul dominio impone che un dato attributo A cada in uno specifico $Dom(A)$ determinato secondo le necessità evidenziate. Si consideri per esempio l'attributo *Voto*, il suo valore deve essere contenuto nell'intervallo [0, 10]. Si è quindi introdotto il **CONSTRAINT** *dominio Voto* per garantire che i valori inseriti rispettino il vincolo individuato.

```
-- 6. Domini per gli attributi
CREATE DOMAIN dominio_voto AS INT
CONSTRAINT range CHECK (VALUE BETWEEN 0 AND 10); /* [7] */
```

2. Vincoli di chiave primaria

Una chiave primaria è un insieme di attributi K che identifica univocamente ogni tupla di una relazione R. I vincoli associati a tale concetto impongono:

- **Unicità:** Non possono essere presenti delle tuple con lo stesso valore di chiave primaria.

- **Non nullità:** Nessun attributo della chiave primaria può assumere valore NULL.

Tali vincoli sono stati implementati tramite l'utilizzo dell'apposita clausola **PRIMARY KEY**, la quale è capace di assicurare le suddette proprietà caratteristiche di una chiave primaria.

E' possibile inoltre definire dei **vincoli di chiave alternativa** tramite la keyword **UNIQUE**, che permette di specificare che l'attributo (o gli attributi) devono presentare valore univoco.

3. Vincoli di integrità referenziale

L'integrità referenziale assicura che le relazioni tra tabelle siano coerenti tramite chiavi esterne. Formalmente: se R_1 ha una chiave primaria K e R_2 ha una chiave esterna $F K$ che fa riferimento a K , allora ogni valore di $F K$ in R_2 deve esistere come valore corrispondente di K in R_1 , oppure può essere NULL se permesso.

Per ogni attributo di una tabella coinvolta in una relazione che rappresenta una **chiave esterna** (**FOREIGN KEY**) ne è stato indicato il corrispondente (**REFERENCES**). Quest'ultimo in particolare rappresenta l'attributo **chiave primaria** dell'altra entità coinvolta nella medesima relazione.

4. Vincoli sui valori NULL e valori predefiniti

L'inserimento di **NOT NULL** nella creazione di una tabella serve a definire un vincolo che **impedisce l'inserimento di valori nulli** in specifiche colonne.

Di seguito le motivazioni che hanno portato alla specifica di tale clausola con relativi esempi di contesti d'implementazione:

- **Garantire l'integrità dei dati**
Alcune informazioni sono essenziali per il funzionamento dell'applicazione. Ad esempio, nel presente schema:
 - a. Il titolo di un **Hackathon** deve sempre esistere
 - b. Il nome di un **Team** deve essere sempre specificato
- **Evitare errori logici**
Omettendo la clausola **NOT NULL**, potrebbero verificarsi scenari inconsistenti, dove un attributo che deve necessariamente avere un valore è sprovvisto di esso.

L'inserimento di **DEFAULT** invece consente di specificare un **valore predefinito** per un attributo, in sostituzione di un dato non inserito o *eliminato*. In particolare, la seconda possibilità è stata introdotta nel sistema per garantire **l'integrità** e la **coerenza** dei dati.

Nel contesto in cui un **Utente** decide di eliminare propri dati dalla piattaforma, si ricadrebbe in uno stato incoerente della base di dati se procedessimo all'eliminazione di tutti i riferimenti ad esso (la clausola ON DELETE CASCADE). E' quindi stato introdotto un **utente fantoccio** per ovviare al problema della coerenza interna tra informazioni, ricollegando ogni **Utente** non più presente nel sistema a questo nuovo record fittizio.

5. Vincoli sulle tuple

I vincoli sulle tuple sono condizioni che devono essere soddisfatte da ogni singola tupla di una relazione.

Questi vincoli possono coinvolgere uno o più attributi della stessa tupla.

Un esempio di vincolo individuato appartenente a tale categoria corrisponde al **CONSTRAINT** denominato **dateCorrette** ([2]), il quale verifica per ogni tupla la relazione che intercorre tra i valori degli attributi *data_apertura_iscrizioni*, *data_chiusura_iscrizioni*, *data_fine*, *data_inizio* della tabella *Hackathon*, verificando che essi siano coerenti con la semantica che descrivono

Dettagli implementativi e considerazioni

- I campi SERIAL garantiscono la gestione automatica di chiavi primarie progressive per identificatori numerici.
- L'ordine di inserimento dei dati deve seguire la stessa sequenza gerarchica utilizzata per la creazione delle tabelle, per assicurarsi di rispettare le **dipendenze** presenti nel sistema

4.3 Implementazione di altri Vincoli — Triggers

Introduzione

La restante parte di vincoli individuati deve essere necessariamente implementata tramite un costrutto più potente di un'espressione booleana o di una clausola durante la creazione delle tabelle: i **trigger**.

Definizione: un **trigger** è un tipo di funzione conservata nel DBMS e innescata da uno specifico evento definito dal trigger stesso, grazie al quale è possibile automatizzare processi di controllo e verifica dei dati nel sistema.

Un esempio di vincoli la cui implementazione richiede l'utilizzo di trigger sono i **vincoli interrelazionali** per i quali è necessario gestire le logiche complesse e operazioni automatiche che richiedono l'esecuzione di codice rispetto a eventi DML su più tabelle.

Implementazione dei Trigger

4.3.1 unique_team_name

Questo trigger si occupa di assicurare che in una data competizione un team non abbia lo stesso nome di un altro. Questo rispetta il vincolo di chiave naturale ([11]) locale rispetto ad un hackathon, presente nelle analisi precedenti.

```
CREATE OR REPLACE FUNCTION unique_team_name_f()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
-- Controlla se esiste gia' un team con lo stesso nome
    IF EXISTS (SELECT 1
                FROM Team
                WHERE nome = NEW.nome AND id_hackathon =
                NEW.id_hackathon)
    THEN
        RAISE EXCEPTION 'Un team con il nome "%" e" gia" esistente',
        NEW.nome;
    END IF;
    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER unique_team_name_t
BEFORE INSERT OR UPDATE OF nome ON Team
FOR EACH ROW
EXECUTE FUNCTION unique_team_name_f();
```

4.3.2 double_role

Questo trigger implementa il vincolo [8], garantendo che prima dell'inserimento una nuova registrazione di un partecipante, esso non stia già ricoprendo un altro ruolo. In particolare, questo è l'unico controllo necessario, poiché una simile verifica a livello database su Organizzatore o Giudice sarebbe superflua (come spiegato nella sezione 4.1).

```
CREATE OR REPLACE FUNCTION double_role_f()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    user_to_check Utente.username%TYPE;
    org_user      Utente.username%TYPE;
    judge_user    Utente.username%TYPE;

    judges_cursor CURSOR FOR (SELECT username
                              FROM Selezione NATURAL JOIN Giudice
                              WHERE id_hackathon = NEW.id_hackathon);
BEGIN
    -- Estraggo l'username
    SELECT username INTO user_to_check
    FROM Partecipante
    WHERE id_partecipante = NEW.id_partecipante;

    -- Prendo l'organizzatore
    SELECT username INTO org_user
    FROM Organizzatore NATURAL JOIN Hackathon
    WHERE id_hackathon = NEW.id_hackathon;

    -- Controllo prima se e' organizzatore
    IF org_user = user_to_check THEN
        RAISE EXCEPTION 'L"organizzatore non puo" partecipare alla
        competizione';
    END IF;

    -- Con questo controllo ogni giudice
    OPEN judges_cursor;
    LOOP
```

```

EXIT WHEN judges_cursor%NOTFOUND;
FETCH judges_cursor INTO judge_user;
IF judge_user = user_to_check THEN
    RAISE EXCEPTION 'Un giudice non puo' partecipare alla competizione';
END IF;
END LOOP;
CLOSE judges_cursor;
RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER double_role_t
BEFORE INSERT ON Registrazione
FOR EACH ROW
EXECUTE FUNCTION double_role_f();

```

4.3.3 cech_complete_examination

La seguente funzione trigger si occupa di verificare che un dato giudice abbia esaminato tutti i documenti di un dato team prima di poter inserire una votazione per esso (vincolo [12]).

```

CREATE OR REPLACE FUNCTION check_complete_examination_f()
RETURNS trigger
LANGUAGE plpgsql
AS $$
DECLARE
    total_docs INT;
    examined_docs INT;
BEGIN
    -- Numero totale di documenti del team
    SELECT COUNT(id_documento) INTO total_docs
    FROM Documento
    WHERE id_team = NEW.id_team;

    -- Numero di documenti del team esaminati dal giudice
    SELECT COUNT(d.id_documento) INTO examined_docs
    FROM Documento d NATURAL JOIN Esaminazione e
    WHERE d.id_team = NEW.id_team AND e.id_giudice = NEW.id_giudice;

    -- Controllo di completezza
    IF total_docs <> examined_docs THEN
        RAISE EXCEPTION 'Non hai esaminato tutti i documenti del team';
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER check_complete_examination_t
BEFORE INSERT ON Votazione

```

```
FOR EACH ROW  
EXECUTE FUNCTION check_complete_examination_f();
```

4.3.4 check_unique_document

Questo trigger controlla che nel contesto di un singolo hackathon un documento non presenti la stessa coppia di titolo e contenuto. Questo garantisce diversità e riconoscibilità del progresso all'interno della stessa competizione (vincolo [13]).

```
CREATE OR REPLACE FUNCTION check_unique_document_f()  
RETURNS trigger  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_title    Documento.titolo%TYPE;  
    v_content  Documento.contenuto%TYPE;  
    v_hack     Hackathon.id_hackathon%TYPE;  
    all_docs   REFCURSOR;  
BEGIN  
    -- Estraggo l'hackathon di riferimento  
    SELECT id_hackathon INTO v_hack  
    FROM Documento NATURAL JOIN Team  
    WHERE id_documento = NEW.id_documento;  
  
    -- Cursore per i dati di tutti i documenti di quell'hackathon  
    OPEN all_docs FOR (SELECT titolo, contenuto  
                        FROM Documento NATURAL JOIN Team  
                        WHERE id_hackathon = v_hack);  
  
    -- Se trovo coppie uguali il vincolo e' violato  
    LOOP  
        EXIT WHEN all_docs%NOTFOUND;  
        FETCH all_docs INTO v_title, v_content;  
        IF v_title = NEW.titolo AND v_content = NEW.contenuto THEN  
            RAISE EXCEPTION 'Il titolo e il commento devono essere univoci';  
        END IF;  
    END LOOP;  
END;
```

```
in questa competizione';
    END IF;
END LOOP;
CLOSE all_docs;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER check_unique_document_t
BEFORE INSERT ON Documento
FOR EACH ROW
EXECUTE FUNCTION check_unique_document_f();
```

4.4 Funzioni e procedure

Definizioni:

(1) Una procedura è un blocco di codice non anonimo identificato da un nome il quale può essere richiamato più volte da altri moduli del programma.

(2) Una funzione è concettualmente simile a una procedura, ma si distingue da essa in quanto caratterizzata da un valore di ritorno. Tale valore può essere utilizzato all'interno di espressioni o istruzioni SQL, compatibilmente con le regole imposte dal sistema di gestione della base di dati.

Introduzione

L'uso congiunto di procedure e funzioni consente di ottenere un'organizzazione del codice più chiara, riutilizzabile e manutenibile. Tuttavia, per sfruttarne le potenzialità, è necessario un linguaggio che permetta di integrarli con le operazioni tipiche delle basi di dati. Ciò è possibile solo tramite PL/SQL, un linguaggio che permette l'introduzione dei costrutti classici della programmazione procedurale in SQL (l.p. dichiarativo). Estendendo così quest'ultimo infatti è stato possibile utilizzare:

- Costrutti condizionali
- Costrutti iterativi
- Gestione delle eccezioni

Implementazione delle procedure

4.4.1 start_hackathon

La seguente procedura si occupa di verificare le condizioni necessarie affinché un hackathon possa partire con successo. In particolare, verifica la validità del vincolo [9] oltre a controllare che la data di inizio sia corretta con quella al momento del lancio della procedura (vincolo [15]). Si è scelto di eliminare direttamente l'istanza di hackathon in questione poiché sarebbe improprio preservare una competizione del quale dovremmo cambiare le informazioni più significative, a fronte di una più semplice riformazione dell'evento in sé.

```
CREATE OR REPLACE PROCEDURE start_hackathon (  
    IN in_hack Hackathon.id_hackathon%TYPE)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    start_date      Hackathon.data_inizio%TYPE;  
    teams_threshold INT;  
BEGIN  
    -- Controllo se la data e' corretta  
    SELECT data_inizio INTO start_date  
    FROM Hackathon  
    WHERE id_hackathon = in_hack;  
  
    IF CURRENT_DATE < start_date THEN  
        RAISE EXCEPTION 'E' troppo presto!';  
    ELSIF CURRENT_DATE > start_date THEN  
        CALL delete_hackathon(in_hack);  
        RAISE EXCEPTION 'Hai avviato troppo tardi!';  
    END IF;  
  
    -- Controllo se ci sono almeno 2 team (vincolo [9])  
    SELECT COUNT(id_team) INTO teams_threshold  
    FROM Hackathon NATURAL JOIN Team  
    WHERE id_hackathon = in_hack;  
  
    IF teams_threshold < 2 THEN  
        CALL delete_hackathon(in_hack);  
        RAISE EXCEPTION 'Non ci sono abbastanza Team iscritti.';  
    END IF;  
END;  
$;
```

4.4.2 authenticate

La seguente procedura consente di identificare il ruolo di un **Utente** all'interno dell'hackathon specificato come parametro di input, così da limitare le operazioni sensibili — come la pubblicazione della *descrizione_problema* — ai soli utenti autorizzati. Utilizza due parametri di output per restituire il ruolo di quell'utente e l'eventuale ID ad esso associato.

```
CREATE OR REPLACE PROCEDURE authenticate (  
  IN in_hack      Hackathon.id_hackathon%TYPE,  
  IN in_username  Utente.username%TYPE,  
  OUT o_role      INT,  
  OUT o_id        INT)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  o_role := 0;  
  o_id   := 0;  
  
  -- ^ Valori di default  
  
  SELECT id_giudice INTO o_id  
  FROM Selezione NATURAL JOIN Giudice  
  WHERE id_hackathon = in_hack AND username = in_username;  
  
  IF FOUND THEN  
    o_role := 1;  
  END IF;  
  
  IF o_id IS NULL THEN  
    SELECT id_organizzatore INTO o_id  
    FROM Hackathon NATURAL JOIN Organizzatore  
    WHERE id_hackathon = in_hack AND username = in_username;  
  
    IF FOUND THEN  
      o_role := 2;  
    END IF;  
  END IF;  
  
  IF o_id IS NULL THEN  
    SELECT id_partecipante INTO o_id  
    FROM Registrazione NATURAL JOIN Partecipante  
    WHERE id_hackathon = in_hack AND username = in_username;  
  
    IF FOUND THEN  
      o_role := 3;  
    END IF;  
  END IF;  
  
  IF o_role = 0 THEN  
    RAISE EXCEPTION 'Non fai parte di questa competizione.';  
  END IF;  
END;
```

```
$$;
```

4.4.3 new_user

Procedura per l'inserimento in piattaforma di un nuovo utente.

```
CREATE OR REPLACE PROCEDURE new_user (  
    IN v_username Utente.username%TYPE,  
    IN v_nome     Utente.nome%TYPE,  
    IN v_cognome  Utente.cognome%TYPE,  
    IN v_password Utente.password%TYPE)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO Utente VALUES (v_username, v_nome, v_cognome,  
v_password);  
EXCEPTION  
    WHEN unique_violation THEN  
        -- Propago l'errore con un messaggio esplicativo  
        RAISE EXCEPTION 'Questo nome utente non e' disponibile';  
END;  
$$;
```

4.4.4 subscribe

La seguente procedura consente agli utenti di prendere parte agli hackathon scelti, verificando la conformità della data attuale con quelle imposte dalla competizione (vincolo [14]) ed eventualmente creando un "account" da partecipante per concedere l'accesso a quel ruolo per l'utente interessato. Viene inoltre creato un team di default nel quale il nuovo partecipante viene inserito automaticamente.

```
CREATE OR REPLACE PROCEDURE subscribe (  
    IN in_hack    Hackathon.id_hackathon%TYPE,  
    IN in_username Utente.username%TYPE)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    start_sub_date    Hackathon.data_apertura_iscrizioni%TYPE;  
    end_sub_date      Hackathon.data_chiusura_iscrizioni%TYPE;  
    in_id_partecipante Partecipante.id_partecipante%TYPE;  
    max_subs          Hackathon.max_iscritti%TYPE;  
    actual_subs        INT;  
    new_team          Team.id_team%TYPE;  
    name              Team.nome%TYPE;  
BEGIN  
    -- Verifico che la data attuale sia coerente con le date da rispettare  
    SELECT data_apertura_iscrizioni INTO start_sub_date  
    FROM Hackathon  
    WHERE id_hackathon = in_hack;  
  
    IF CURRENT_DATE < start_sub_date THEN  
        RAISE EXCEPTION 'Non e" ancora possibile iscriversi a questo  
hackathon';  
    END IF;  
  
    SELECT data_chiusura_iscrizioni INTO end_sub_date  
    FROM Hackathon  
    WHERE id_hackathon = in_hack;  
  
    IF CURRENT_DATE > end_sub_date THEN  
        RAISE EXCEPTION 'Non e" piu" possibile iscriversi a questo  
hackathon';  
    END IF;  
  
    -- Verifico che il partecipante esista  
    SELECT id_partecipante INTO in_id_partecipante  
    FROM Partecipante  
    WHERE username = in_username;  
  
    IF NOT FOUND THEN  
        INSERT INTO Partecipante (username) VALUES (in_username)  
RETURNING id_partecipante INTO in_id_partecipante;
```

```

END IF;

-- Verifico che il partecipante non sia gia' registrato
SELECT 1
FROM Registrazione
WHERE id_partecipante = in_id_partecipante AND id_hackathon = in_hack;

IF FOUND THEN
    RAISE EXCEPTION 'Sei gia" iscritto a questo hackathon';
END IF;

-- Verifico che sia possibile iscriversi entro i limiti predefiniti
SELECT max_iscritti INTO max_subs
FROM Hackathon
WHERE id_hackathon = in_hack;

SELECT COUNT(*) INTO actual_subs
FROM Registrazione
WHERE id_hackathon = in_hack;

IF (actual_subs+1) > max_subs THEN
    RAISE EXCEPTION 'Il limite massimo di iscrizioni e" stato raggiunto';
END IF;

-- Inserisco la registrazione e il relativo team di default
INSERT INTO Registrazione VALUES(in_id_partecipante, in_hack,
CURRENT_DATE);

name := FORMAT('Team di %L', in_username);

INSERT INTO Team (nome, numero_membri, id_hackathon)
VALUES (name, 1, in_hack) RETURNING id_team INTO new_team;

INSERT INTO Partecipazione VALUES (new_team, in_id_partecipante);
END;
$$;

```

4.4.5 end_hackathon

Procedura per gestire correttamente tutte le operazioni gestionali per la corretta chiusura di un hackathon. Verifica che siano presenti votazioni per tutti i team da tutti i giudici — inserendo quelle mancanti di default — e restituisce la classifica finale.

```
CREATE OR REPLACE FUNCTION end_hackathon(
    in_hack Hackathon.id_hackathon%TYPE)
RETURNS refcursor
LANGUAGE plpgsql
AS $$
DECLARE
    winners      REFCURSOR;
    all_judges    CURSOR FOR (SELECT id_giudice
                                FROM Selezione
                                WHERE id_hackathon = in_hack);
    one_judge     Giudice.id_giudice%TYPE;
    all_teams     CURSOR FOR (SELECT id_team
                                FROM Team
                                WHERE id_hackathon = in_hack);
    one_team      Team.id_team%TYPE;
BEGIN
    OPEN all_judges;
    OPEN all_teams;

    LOOP
        -- Controllo che siano presenti tutte le votazioni necessarie
        FETCH all_judges INTO one_judge;
        EXIT WHEN NOT FOUND;

        LOOP
            FETCH all_teams INTO one_team;
            EXIT WHEN NOT FOUND;

            CALL check_complete_grading(one_judge, one_team);
        END LOOP;
    END LOOP;

    -- Produco la classifica finale
    SELECT scoreboard (in_hack) INTO winners;

    CLOSE all_judges;
    CLOSE all_teams;

    RETURN winners;
END;
$;
```

4.4.6 delete_hackathon

Procedura per l'eliminazione di un dato hackathon.

```
CREATE OR REPLACE PROCEDURE delete_hackathon (  
    IN id_hack_to_del Hackathon.id_hackathon%TYPE)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    DELETE FROM Hackathon WHERE id_hackathon = id_hack_to_del;  
END;  
$$;
```

4.4.7 atleast_1_progres

Tale procedura è utile a verificare che al termine di un hackathon ogni team abbia pubblicato almeno un progresso (vincolo [16]). In caso contrario, verrà assegnato automaticamente un voto pari a zero e inserito il relativo record nella tabella Votazione. Tale convenzione è stata adatta per rispettare l'

```
CREATE OR REPLACE PROCEDURE atleast_1_progress (  
    IN in_hack Hackathon.id_hackathon%TYPE)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    -- Corsore per tutti i team dell'hackathon interessato  
    teams_cursor CURSOR FOR (SELECT id_team  
                                FROM Team  
                                WHERE id_hackathon = in_hack);  
    current_team Team.id_team%TYPE;  
    all_judges REFCURSOR;  
    current_judge Giudice.id_giudice%TYPE;  
BEGIN  
    OPEN teams_cursor;  
  
    -- Controllo se tutti i team hanno almeno un documento pubblicato  
    LOOP  
        EXIT WHEN teams_cursor%NOTFOUND;  
        FETCH teams_cursor INTO current_team;  
  
        IF current_team NOT IN (SELECT id_team  
                                FROM Documento) THEN  
            -- Se non c'e' nemmeno un documento allora la votazione e'  
            automaticamente pari a 0  
            OPEN all_judges FOR (SELECT id_giudice  
                                    FROM Selezione  
                                    WHERE id_hackathon = in_hack);  
            LOOP  
                EXIT WHEN all_judges%NOTFOUND;
```

```

        FETCH all_judges INTO current_judge;
        INSERT INTO Votazione VALUES (current_judge, current_team, 0);
    END LOOP;
    CLOSE all_judges;
END IF;
END LOOP;

CLOSE teams_cursor;
END;
$$;

```

4.4.8 publish-problem

Questa procedura permette a uno dei giudici di pubblicare la descrizione al problema da affrontare nell'hackathon di cui sono esaminatori.

```

CREATE OR REPLACE PROCEDURE publish_problem (
    in_id_hackathon Hackathon.id_hackathon%TYPE,
    in_descrizione Hackathon.descrizione_problema%TYPE)
LANGUAGE plpgsql
AS $$
BEGIN
    -- Aggiorna la descrizione del problema
    UPDATE Hackathon
    SET descrizione_problema = in_descrizione
    WHERE id_hackathon = in_id_hackathon;
END;
$$;

```

4.4.9 join_team

Questa procedura si occupa di gestire l'entrata di un partecipante in un team diverso da quello assegnatogli di default. In particolare si verificano in primis i casi base che possono produrre errori logici, come il tentativo di partecipazione al team di cui si fa già parte. Successivamente la procedura si occupa di verificare che sia effettivamente possibile unirsi al team controllando la coerenza con la capienza massima espressa dall'attributo `max_dim_team` (vincolo [17]). Una volta garantite tutte le ipotesi di partecipazione, si procede alla sostituzione del record vecchio con quello nuovo, incrementando l'attributo `numero_membri` del team a cui ci si vuole unire.

```

CREATE OR REPLACE PROCEDURE join_team (
    IN in_username Partecipante.username%TYPE,
    IN in_id_team Team.id_team%TYPE)
LANGUAGE plpgsql
AS $$
DECLARE
    in_id_partecipante Partecipante.id_partecipante%TYPE;
    already_member INT;

```



```

max_members      Hackathon.max_dim_team%TYPE;
actual_members    Team.numero_membri%TYPE;
old_members       Team.numero_membri%TYPE;
old_team          Team.id_team%TYPE;
v_hack            Hackathon.id_hackathon%TYPE;
start_date        Hackathon.data_inizio%TYPE;
BEGIN
-- Verifica che l'utente esista e sia partecipante
SELECT id_partecipante INTO in_id_partecipante
FROM Partecipante
WHERE username = in_username;

IF NOT FOUND THEN
    RAISE EXCEPTION 'Partecipante "%" non trovato', in_username;
END IF;

-- Verifica che non sia troppo tardi per cambiare team
SELECT id_hackathon INTO v_hack
FROM Team
WHERE id_team = in_id_team;

SELECT data_inizio INTO start_date
FROM Hackathon
WHERE id_hackathon = v_hack;

IF start_date < CURRENT_DATE THEN
    RAISE EXCEPTION 'E" troppo tardi per cambiare team!';
END IF;

-- Verifica che il partecipante non sia gia' nel team
SELECT COUNT(*) INTO already_member
FROM Partecipazione
WHERE id_team = in_id_team AND id_partecipante = in_id_partecipante;

IF already_member > 0 THEN
    RAISE EXCEPTION 'Partecipante "%" e" gia" membro del team %',
in_username, in_id_team;
END IF;

-- Verifica che il team non sia gia' alla capienza massima
SELECT max_dim_team INTO max_members
FROM Hackathon NATURAL JOIN Team
WHERE id_team = in_id_team;

SELECT numero_membri INTO actual_members
FROM Team
WHERE id_team = in_id_team;

IF (actual_members+1) > max_members THEN
    RAISE EXCEPTION 'Il team ha gia" raggiunto la dimensione massima
concessa';
END IF;

-- Gestisco il vecchio team prima di inserire il nuovo record
SELECT id_team INTO old_team
FROM Partecipazione NATURAL JOIN Team
WHERE id_partecipante = in_id_partecipante AND id_hackathon = v_hack;

SELECT numero_membri INTO old_members
FROM Team
WHERE id_team = old_team;

```

```

IF old_members-1 = 0 THEN
    DELETE FROM Team WHERE id_team = old_team;
ELSE
    UPDATE Team SET numero_membri = (numero_membri-1) WHERE id_team =
old_team;
    DELETE FROM Partecipazione WHERE id_partecipante = in_id_partecipante
AND id_team = old_team;
END IF;
-- Inserisce la partecipazione e incrementa il contatore
INSERT INTO Partecipazione(id_team, id_partecipante) VALUES (in_id_team,
in_id_partecipante);
UPDATE Team SET numero_membri = numero_membri+1 WHERE id_team =
in_id_team;
END;
$$;

```

4.4.10 grade_team

Procedura per inserire correttamente una votazione per un dato team. Sfrutta il trigger check_complete_examination per verificare la coerenza della nuova operazione.

```

CREATE OR REPLACE PROCEDURE grade_team(
    in_id_giudice Giudice.id_giudice%TYPE,
    in_id_team    Team.id_team%TYPE,
    in_voto      INT)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO Votazione VALUES (in_id_giudice, in_id_team, in_voto);
END;
$$;

```

4.4.11 check_complete_grading

Questa procedura è tale da garantire che alla fine dell'competizione ogni team abbia le votazioni necessarie per la corretta conclusione della stessa (vincolo [18]). Il voto predefinito assegnatogli in assenza di quello dei giudici è 6

```
CREATE OR REPLACE PROCEDURE check_complete_grading(  
    in_id_giudice Giudice.id_giudice%TYPE,  
    in_id_team    Team.id_team%TYPE)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- Controlla se il giudice ha già votato il team  
    IF NOT EXISTS (SELECT 1  
                   FROM Votazione  
                   WHERE id_giudice = in_id_giudice  
                   AND id_team = in_id_team)  
    THEN  
        -- Inserisce voto predefinito = 6 se il giudice non ha votato il team  
        INSERT INTO Votazione(id_giudice, id_team, voto) VALUES  
(in_id_giudice, in_id_team, 6);  
    END IF;  
END;  
$$;
```

4.4.12 scoreboard

La seguente funzione consente di visualizzare la classifica finale di un singolo hackathon, mostrando i vincitori e tutti i relativi voti finali.

```
CREATE OR REPLACE FUNCTION scoreboard (  
    IN in_hack Hackathon.id_hackathon%TYPE)  
RETURNS refcursor  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    winners REFCURSOR;  
BEGIN  
    -- Estraggo dalla classifica globale i team dell'hackathon di riferimento  
    OPEN winners FOR (SELECT nome_team, voto_finale  
                       FROM overall_ranking  
                       WHERE id_hackathon = in_hack);  
  
    RETURN winners;  
END;  
$$;
```

4.4.13 invite_judge

La seguente procedura interna si occupa di invitare singolarmente un giudice alla competizione specificata.

```
CREATE OR REPLACE PROCEDURE invite_judge(  
    IN in_username Utente.username%TYPE,  
    IN in_hack      Hackathon.id_hackathon%TYPE,  
    IN in_org       Organizzatore.id_organizzatore%TYPE)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_judge Giudice.id_giudice%TYPE;  
BEGIN  
    SELECT id_giudice INTO v_judge  
    FROM Giudice  
    WHERE username = in_username;  
  
    -- Verifico l'esistenza del giudice  
    IF NOT FOUND THEN  
        INSERT INTO Giudice (username) VALUES (in_username) RETURNING  
id_giudice INTO v_judge;  
    END IF;  
  
    INSERT INTO Selezione VALUES (in_hack, in_org, v_judge);  
END;  
$$;
```

4.4.14 publish_progres

Questa procedura permette ai team di pubblicare un documento sulla piattaforma per aggiornare i giudici dei loro progressi. Sfrutta il trigger *check_unique_document* per verificare la validità del documento appena creato.

```
CREATE OR REPLACE PROCEDURE publish_progress (  
    IN in_team      Team.id_team%TYPE,  
    IN in_content   Documento.contenuto%TYPE,  
    IN in_title     Documento.titolo%TYPE)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    -- Inserisco il documento con i relativi dati  
    INSERT INTO Documento (contenuto, titolo, id_team) VALUES (in_content,  
in_title, in_team);  
END;  
$$;
```

4.4.15 add_hackathon

Procedura per la creazione di un nuovo hackathon all'interno del sistema. Una volta individuati i parametri dell'competizione, la procedura assegna l'organizzatore corretto (o ne crea uno se necessario) e registra la selezione dei giudici nella base di dati.

```
CREATE OR REPLACE PROCEDURE add_hackathon (  
    IN v_title          Hackathon.titolo%TYPE,  
    IN v_location       Hackathon.sede%TYPE,  
    IN v_start_d        Hackathon.data_inizio%TYPE,  
    IN v_end_d          Hackathon.data_fine%TYPE,  
    IN v_start_sub      Hackathon.data_apertura_iscrizioni%TYPE,  
    IN v_end_sub        Hackathon.data_chiusura_iscrizioni%TYPE,  
    IN v_max_sub        Hackathon.max_iscritti%TYPE,  
    IN v_max_team_size  Hackathon.max_dim_team%TYPE,  
    IN v_username       Utente.username%TYPE,  
    IN v_judges         TEXT)  
-- Testo formattato come 'mario.rossi,luca.bianchi,'  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    duration  INT;  
    v_org     Organizzatore.id_organizzatore%TYPE;  
    v_hack    Hackathon.id_hackathon%TYPE;  
    pos       INT;  
    one_judge Utente.username%TYPE;  
BEGIN  
    -- Calcolo la durata  
    duration := v_end_d - v_start_d;  
  
    -- Verifico l'esistenza dell'organizzatore  
    SELECT id_organizzatore INTO v_org  
    FROM Organizzatore  
    WHERE username = v_username;  
  
    IF NOT FOUND THEN  
        INSERT INTO organizzatore (username) VALUES (v_username)  
RETURNING id_organizzatore INTO v_org;  
    END IF;  
  
    -- Creo l'hackathon  
    INSERT INTO Hackathon (  
        titolo, sede, durata, data_inizio, data_fine,  
        data_apertura_iscrizioni, data_chiusura_iscrizioni,  
        max_iscritti, max_dim_team, id_organizzatore)  
    VALUES (v_title, v_location, duration, v_start_d, v_end_d,  
            v_start_sub, v_end_sub, v_max_sub, v_max_team_size, v_org)  
RETURNING id_hackathon INTO v_hack;  
  
    -- Inserisco le relative selezioni dei giudici  
    LOOP  
        pos := INSTR(v_judges, ',');  
        EXIT WHEN pos <= 0;
```

```

        one_judge := SUBSTR(v_judges, 1, pos - 1);
        v_judges := LTRIM(v_judges, one_judge);
        v_judges := LTRIM(v_judges, ',');
        CALL invite_judge(one_judge, v_hack, v_org);
    END LOOP;

EXCEPTION
    WHEN unique_violation THEN
        RAISE EXCEPTION 'In questa sede si e" gia" svolto un hackathon
con questo nome';
END;
$$;

```

4.4.16 delete_user

Procedura per l'eliminazione di un utente dalla piattaforma.

```

CREATE OR REPLACE PROCEDURE delete_user (
    IN username_to_del Utente.username%TYPE)
LANGUAGE plpgsql
AS $$
BEGIN
    -- Verifica se l'utente esiste
    IF NOT EXISTS (SELECT 1
                    FROM Utente
                    WHERE username = username_to_del)
    THEN
        RAISE EXCEPTION 'Utente "%" non trovato', username_to_del;
    END IF;

    -- Elimina solo l'utente (eventuali vincoli ON DELETE CASCADE
    -- gestiranno le dipendenze se definiti sul DB)
    DELETE FROM Utente WHERE username = username_to_del;
END;
$$;

```

4.4.17 popola_database

La procedura **popola_database()** è stata progettata per inserire dati di esempio coerenti in tutte le tabelle del database, rispettando l'ordine gerarchico delle dipendenze e tutti i vincoli definiti.

4.5 Viste

Introduzione

Un ulteriore strumento utile per un utilizzo efficiente del DB, offerto dai DBMS, è quello delle **viste**:

Definizione: Una vista in un sistema di gestione di database relazionale è una tabella virtuale derivata dal risultato di una query SQL che opera su una o più tabelle base del database.

Il compito delle viste è quello di fornire una visione alternativa degli schemi fisici presenti nel sistema, consentendo di nascondere dati per questioni di riservatezza, creare tabelle NON fisiche con informazioni rilevanti operativamente per il DB e molto altro. Di seguito le viste introdotte nel sistema:

Implementazione delle Viste

4.6.1 overall_ranking

Una vista per visualizzare la classifica globale di tutti i team che hanno partecipato ad una competizione.

```
CREATE OR REPLACE VIEW overall_ranking AS
SELECT nome AS nome_team, ROUND(AVG(voto), 2) AS voto_finale,
id_hackathon
FROM Votazione NATURAL JOIN Team
GROUP BY nome_team, id_hackathon
ORDER BY voto_finale DESC;
```

5 Repository GitHub

Il presente documento è disponibile su GitHub al seguente link:

github.com/fiammass/Progetto_Hackathon

La repository contiene:

- Il file per la creazione completa del database
- Questa documentazione di progetto